Gate 04 Reconciliation: Django Admin Configuration

Field	Value
Gate	04 — Django Admin Configuration
Spec version	Original (pre-implementation)
Implementation date	2026-04-16
Reconciled by	Claude Code
Branch	gate/04-django-admin

Summary

Gate 04 was implemented as specced with all model registrations, list configurations, inlines, and admin branding matching the spec. Security and code quality reviews during implementation identified several vulnerabilities and improvements that led to additions beyond the original spec — most notably queryset-level institution scoping, privilege escalation prevention on UserAdmin, IDOR protection in save_model, and `is_sensitive` write restrictions.

Acceptance Criteria Status

#	Criterion (from spec)	Status	Notes
1	All models load in Django Admin without errors	Implemented as specced	12 model admins registered, all changelist smoke tests pass
2	Institution-scoped write protection tests pass for ExSituPopulation	Modified	Spec called for `get_queryset()` scoping — implemented that plus `has_change_permission`, `has_delete_permission`, and `save_model` with IDOR protection
3	All `list_filter` values return correct filtered querysets	Implemented as specced	All filter configurations match spec
4	SpeciesLocality admin loads with OSMGeoAdmin map widget	Implemented as specced	Uses `GISModelAdmin` (Django 5.x successor to `OSMGeoAdmin`)
5	SpeciesLocalityInline renders on Species admin change form	Implemented as specced	`extra=0`, `show_change_link=True` as specced
6	Watershed and ProtectedArea admin pages load with read-only geometry	Implemented as specced	`geometry` in `readonly_fields` for both
7	Invoke code-quality-reviewer	Implemented as specced	Completed; all critical/important findings addressed

8	Invoke security-reviewer	Implemented as specced	Completed; all critical/high findings addressed

User Story Status

Story	Title	Status	Notes
Species undescribed morphospecies	Admin accepts null authority/year_described	Implemented as specced	Model allows null; admin has no extra validation blocking it
Species provisional name search	Search by provisional_name	Implemented as specced	`provisional_name` in `search_fields`
ConservationAssessment pending review filter	Filter by review_status	Implemented as specced	`review_status` in `list_filter`
ConservationAssessment status change	Change review_status and save	Implemented as specced	Standard admin form
ExSituPopulation census count	Tier 3 records HoldingRecord for own institution	Modified	Reporter auto-set via `save_formset` (not `save_model` — inlines don't use `save_model`)
ExSituPopulation institution scoping	Tier 3 blocked from other institution's save	Modified	Strengthened: `get_queryset` scoping hides other institutions entirely; `save_model` checks original DB value to prevent IDOR
ExSituPopulation holding records	Historical census records in chronological order	Implemented as specced	HoldingRecord model has `ordering = ["-date"]`
SpeciesLocality map widget	OSMGeoAdmin for point placement	Implemented as specced	`GISModelAdmin` provides OpenLayers map widget
SpeciesLocality inline on Species	Inline with show_change_link	Implemented as specced	
User tier elevation	Changing access_tier takes immediate effect	Implemented as specced	Token auth reads `access_tier` from DB on each request

Deviations

GISModelAdmin instead of OSMGeoAdmin

- **Spec said:** Use `OSMGeoAdmin` from `django.contrib.gis.admin`
- **Implementation does:** Uses `GISModelAdmin` from `django.contrib.gis.admin`

- **Reason:** `OSMGeoAdmin` was deprecated in Django 4.x and removed. `GISModelAdmin` is its replacement with identical functionality (OpenLayers-based map widget).
- **Impact:** None — same user-facing behavior.

search_fields uses `common_names__name` instead of `commonname__name`

- **Spec said:** `search_fields = ["scientific_name", "provisional_name", "commonname__name"]`
- **Implementation does:** `search_fields = ["scientific_name", "provisional_name", "common_names__name"]`
- **Reason:** Django admin search follows the model's `related_name`, which is `common_names` (set on the `CommonName.species` ForeignKey). Using the db table name `commonname` would not work.
- **Impact:** None — correct Django ORM lookup syntax.

ExSituPopulation queryset scoping added

- **Spec said:** "Tier 3-4 users restricted to records for their affiliated institution (enforced via `get_queryset()` overrides on ModelAdmin subclasses)"
- **Implementation does:** `get_queryset()` filters to user's institution AND `has_change_permission` / `has_delete_permission` check object-level access AND `save_model` verifies original DB value to prevent IDOR
- **Reason:** Security review identified that `has_change_permission` alone was insufficient: (1) list view leaked all records, (2) form-bound institution_id could be manipulated to bypass save_model checks
- **Impact:** Tier 3-4 users now only see their own institution's records in the changelist (stronger than spec, which implied they could see all but only edit their own)

flagged_by made read-only

- **Spec said:** `flagged_by` listed as an editable field in ConservationAssessmentInline
- **Implementation does:** `flagged_by` is in `readonly_fields` on both the inline and standalone admin
- **Reason:** Code quality review identified that making `flagged_by` an arbitrary FK picker allows pointing it at any user, breaking audit integrity. Flagging should be done through a controlled action, not a raw FK picker.
- **Impact:** Admin users can no longer manually set/clear `flagged_by`. A dedicated flagging action should be added post-MVP if needed.

HoldingRecord reporter set via save_formset, not save_model

- **Spec said:** `reporter` is readonly, "set automatically on save, not editable"
- **Implementation does:** Reporter auto-set in `ExSituPopulationAdmin.save_formset()` instead of `HoldingRecordInline.save_model()`
- **Reason:** Code quality review identified that `TabularInline` does not call `save_model` — Django inlines use `save_formset`. The original `save_model` override was a silent no-op.
- **Impact:** None — reporter is now correctly auto-populated on new HoldingRecords.

Additions (not in spec)

EmailBackend extends ModelBackend

- `accounts/backends.py` changed to extend `ModelBackend` instead of a bare class. Without this, `has_perm()` always returned False for non-superuser staff, making Django Admin completely non-

functional for Tier 3-4 users. This was a bug in Gate 03's implementation discovered during Gate 04 testing.

Privilege escalation prevention on UserAdmin

- `get_readonly_fields` makes `access_tier`, `is_active`, `is_staff`, `is_superuser` read-only for non-superusers. Security review identified that any `is_staff` user with `change_user` permission could elevate their own tier or grant themselves superuser status.

is_sensitive restricted to superusers

- `SpeciesLocalityAdmin.get_readonly_fields` and `SpeciesLocalityInline.readonly_fields` prevent non-superusers from toggling `is_sensitive`. Security review identified this as a data integrity risk — toggling it controls whether exact coordinates are exposed to lower-tier users.

list_select_related on multiple admins

- Added to `SpeciesAdmin` (`taxon`), `ConservationAssessmentAdmin` (`species`, `flagged_by`), `TaxonAdmin` (`parent`), `FieldProgramAdmin` (`lead_institution`), `AuditLogAdmin` (`user`). Code quality review identified N+1 query risks on all changelists with FK fields in `list_display`.

AuditLogAdmin registered as fully read-only

- Not in the spec's explicit list of admin registrations, but `AuditLog` exists in the model layer. Registered with all fields read-only and add/change/delete blocked — audit records are system-generated.

TaxonAdmin registered

- Not explicitly in the spec but `Taxon` is an MVP model used by Species. Registered with basic list/filter/search.

institution added to UserAdmin add_fieldsets

- Spec's `add_fieldsets` omitted `institution`. Code quality review noted that creating a Tier 3-4 user without an institution immediately puts them in a broken state for institution-scoped admin access.

Deferred Items

Item	Deferred to	Reason
Django Admin login rate limiting	Post-MVP	Security review H3; requires django-axes or custom AdminSite. API login is already rate-limited.
FieldProgram institution scoping	Post-MVP	Security review M3; spec says Tier 3+ has global read access to field programs
AuditLog field-level redaction	Post-MVP	Security review M4; AuditLog write path not implemented yet (Gate 05)
Non-default admin URL path	Post-MVP/deployment	Security review L4; operational hardening

Custom flagging action for ConservationAssessment	Post-MVP	To replace the now-readonly `flagged_by` field

Technical Decisions Made During Implementation

1. **EmailBackend must extend ModelBackend** — This is a fundamental requirement for Django Admin to work for non-superuser staff. The bare `EmailBackend` from Gate 03 silently broke `has_perm()`, making all model-level permission checks fail. This should be noted in the architecture proposal.

2. **Institution scoping at queryset level is strictly stronger than per-object permission checks** — The spec implied Tier 3-4 could see all records but only edit their own. Implementation scopes the queryset itself, hiding other institutions' records entirely. This is the more secure default and prevents enumeration attacks.

3. **IDOR protection requires checking original DB values** — Django's form binding overwrites `obj` fields before `save_model` runs, so checking `obj.institution_id` after form processing is insufficient. The implementation re-fetches the original record from the database for update operations.

Spec Updates Needed

- **Gate 04 spec** should note that `OSMGeoAdmin` is now `GISModelAdmin` in Django 5.x
- **Gate 04 spec** should note `common_names__name` (not `commonname__name`) for the search_fields lookup
- **Gate 03 spec** should document that `EmailBackend` must extend `ModelBackend` for Django Admin compatibility
- **Architecture proposal** should document the decision that institution scoping uses queryset-level filtering (not just per-object permission checks)